Dyna Planning Budget and Model Staleness

Status: independent precursor proposal with completed 20-seed extended run; strongest use is as a diagnostic foundation for the integrated Continual Dyna Model Aging study.

Abstract

Dyna-style planning is attractive for a continual agent because it converts ordinary experience into a compact learned model and then uses limited background computation to improve values without collecting extra data. In a changing world, however, a larger planning budget can also spend more computation on model entries that describe the previous world. This proposal asks whether a fixed per-step planning budget creates a tradeoff between pre-change sample efficiency and post-change stale computation. In a continuing gridworld that changes goal and hazard layout midstream, larger planning budgets improve pre-change reward, but keeping the old model after the change leaves a measurable stale-backup signal even when late reward eventually recovers. The result is not that flushing the model is a practical solution; it is an oracle diagnostic showing why model freshness must be studied separately from planning budget.

Standalone Study Summary

This study is deliberately narrower than the full model-aging proposal. It does not introduce a realistic freshness mechanism such as age-weighted sampling or model-error gating. Instead, it establishes the failure mode that makes those mechanisms necessary. The RL problem is a continuing gridworld with no episodic train/test split. The agent learns a tabular action-value function and a one-step transition/reward model from the stream. After each real transition, it performs a fixed number of Dyna backups sampled from the model. The experiment varies planning budget and model handling after an unannounced phase change: the ordinary agent keeps its model, while an oracle diagnostic flushes the model exactly at the change.

Research Motivation

The Alberta Plan gives learned models and background planning a central role in building long-lived agents from ordinary experience. Standard Dyna examples often emphasize sample efficiency in stationary problems: the agent learns a model, performs simulated backups, and improves faster than a model-free learner. A continual agent faces a harder question. Its model is a memory of the stream, and the stream may stop obeying old dynamics or reward consequences. If the agent samples old entries blindly, planning can become a mechanism for preserving obsolete knowledge.

The important Core RL issue is therefore not simply “does planning help?” It is “when the world changes, how should a small online agent allocate limited planning computation across model entries of unequal freshness?” This precursor study isolates the allocation problem before adding more elaborate freshness rules.

Research Question

The main question is: how does planning budget affect the tradeoff between pre-change learning speed and post-change stale computation in a continuing task?

The concrete subquestions are:

• - Does increasing the number of Dyna backups improve reward before the environment changes?

• - After the change, does the keep-model agent continue to perform backups from old-phase model entries?

• - Can an oracle flush separate the effect of model freshness from the effect of planning budget?

• - Do reward curves alone hide a staleness problem that process metrics can reveal?

Alberta Plan Connection

This proposal connects directly to learned models, background planning, continuing control, limited computation, ordinary experience, and temporal uniformity. The model is a learned one-step table, not a replay buffer. The agent does not store a dataset for later offline training; it updates online from each real transition and performs small model-based TD backups under a fixed computation budget.

Related Work

Dyna unifies direct RL, model learning, and planning backups. Prioritized sweeping shows that search control is itself a central design problem, because not all backups are equally useful. Continual RL reframes evaluation around adaptation and nonstationarity rather than final stationary performance. Average-reward and continuing-task work motivates reporting ongoing reward and recovery windows rather than episodic returns alone.

Local references used for this proposal include resources/alberta_plan_related/alberta_plan_2208.11173.pdf, resources/alberta_plan_related/average_reward_learning_planning_2006.16318.pdf, and resources/alberta_plan_related/rethinking_foundations_continual_rl_2504.08161.pdf.

Environment

The environment is a continuing gridworld with a start region, one rewarding goal region, hazards, a small step cost, and no terminal training episode boundary. The agent keeps interacting indefinitely. At the midpoint of the stream, the environment switches phase: the goal and hazards move. This creates a controlled nonstationarity while preserving a simple tabular state-action space.

The extended experiment uses grid size 11 x 11, seeds 0-19, and 20000 online steps. The phase change occurs halfway through the stream. The task is intentionally larger than a two-state toy problem, but still simple enough that every mechanism can be inspected: Q norm, model size, stale-backup rate, and recovery windows are all logged.

Agent And Baselines

All agents use tabular Q-learning from real transitions with epsilon-greedy behavior, alpha = 0.1, gamma = 0.95, and epsilon = 0.1. After each real transition the agent writes the observed next state, reward, and current phase label into a learned model entry for the visited state-action pair. Then it performs 0, 1, 5, or 20 sampled planning backups.

The keep-model condition is the realistic baseline for this precursor: the agent receives no change signal and retains all learned entries. The flush-on-change condition is an oracle diagnostic: it clears the model exactly when the phase

changes. It is not proposed as an implementable continual-learning algorithm. Its role is to reveal how much of the post-change behavior is caused by stale model entries rather than the planning budget itself.

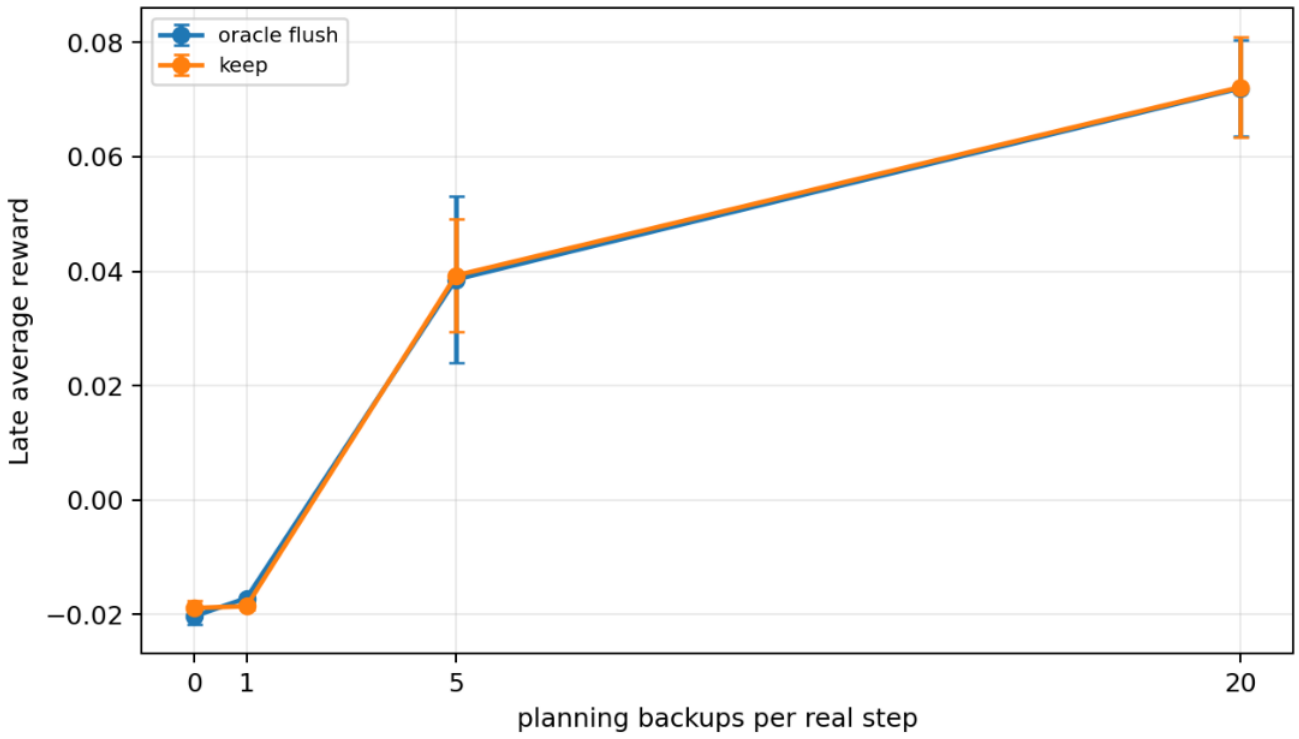
Experimental Design

Current extended run:

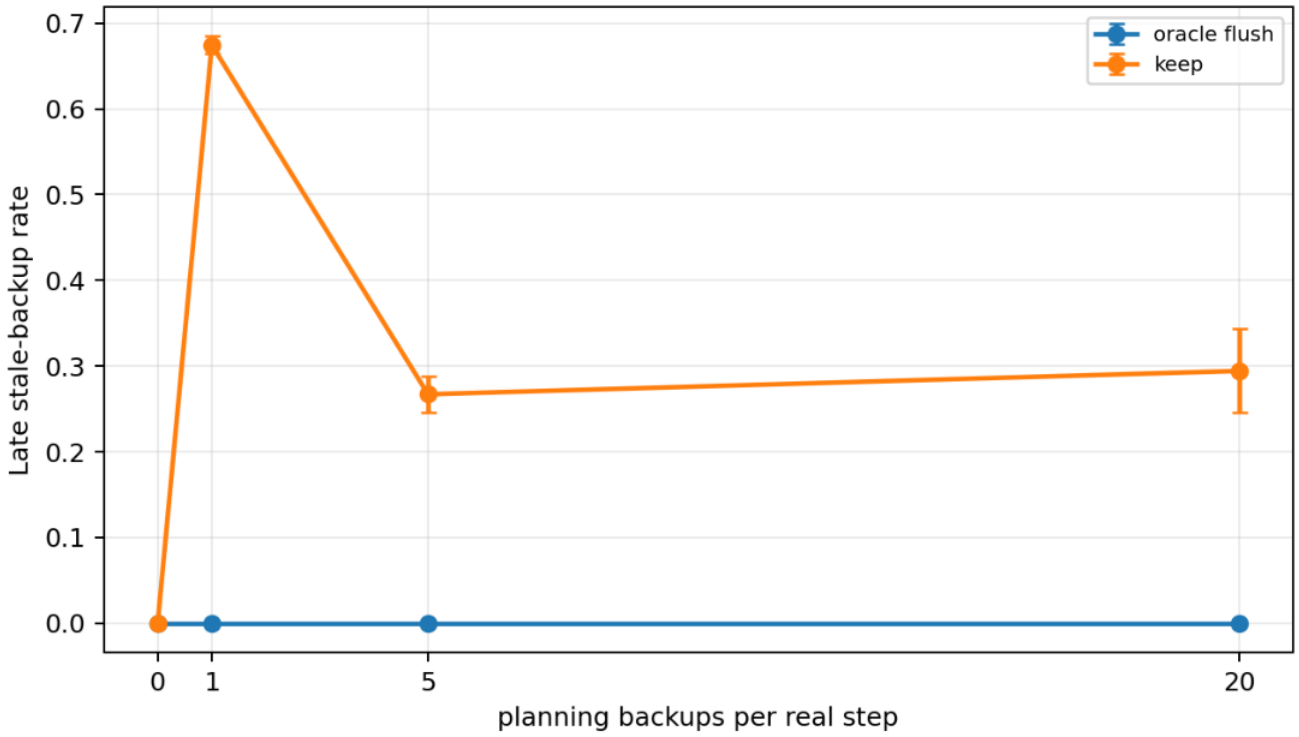
- - Result path: experiments/alberta_core_rl/results/dyna_planning_budget/20260709T024602Z_extended.
- - Seeds: 0-19.
- - Steps per seed/condition: 20000.
- - Grid size: 11 x 11.
- - Planning budgets: 0, 1, 5, 20 backups per real step.
- - Model modes: keep_model and flush_on_change.
- - Metrics: online average reward, stale-backup rate, model size, Q norm, phase, and recovery window.

The primary dependent variable for the research question is not only reward. Reward shows whether the policy recovers; stale-backup rate shows whether the planning computation remains temporally appropriate. A method could recover reward late while still wasting substantial computation on obsolete entries, which would matter for a larger agent with many competing uses of computation.

Results



Late average reward by planning budget and model handling in the 20-seed extended run.



Late stale-backup rate by planning budget and model handling in the 20-seed extended run.

Before the change, planning improves reward. With no planning, pre-change average reward is about 0.070. With one backup it rises to about 0.078, with five backups to about 0.078-0.079, and with twenty backups to about 0.079. The gain saturates quickly, but it confirms that the model is useful when it is fresh.

Immediately after the change, all planning settings suffer. This is expected because the value function and model have been shaped by the previous layout. The important difference is the stale-backup signal. In the keep-model condition, stale-backup rate is high after the switch: at budget 20 it is about 0.749 in the first 250 post-change steps, about 0.910 in the next 250 steps, about 0.835 in the 500-1000 step window, and still about 0.294 +/- 0.049 in the late post-change window. At budget 5, the late stale-backup rate is about 0.267 +/- 0.021; at budget 1, it is about 0.675 +/- 0.010.

Flush-on-change eliminates stale backups by construction. However, this does not make it a final algorithm, and it does not mean the reward story is simple. With budget 20, late reward is about 0.07198 +/- 0.00847 for flush and 0.07217 +/- 0.00870 for keep-model. With budget 5, late reward is about 0.03855 +/- 0.0145 for flush and 0.03924 +/- 0.00981 for keep-model. These late reward numbers show that policy recovery can occur even while the keep-model agent continues to sample some stale entries.

Analysis

The central insight is that reward and planning quality can separate. If one looked only at late reward for budget 20, one might conclude that keeping the model is harmless because the reward matches the oracle flush diagnostic. The stale-backup metric says something different: a nontrivial fraction of background computation is still assigned to old-phase entries. In a small grid this waste may not dominate reward, but in a larger Alberta Plan-style agent with many predictions, options, and planning tasks competing for computation, this would be a serious allocation problem.

The experiment also shows why a larger planning budget is not automatically better. More backups increase the chance of useful model use before the change, but they

also amplify the consequences of poor search control after the change. The right problem formulation is therefore not “choose a budget,” but “learn a search-control distribution that respects model freshness under a budget.”

This is why the proposal should be treated as a precursor to Continual Dyna Model Aging. It supplies the empirical reason to study age-weighted sampling, prediction-error gating, or other online freshness estimates. It should not be packaged as a final claim that flush-on-change wins, because the flush condition uses privileged information and the reward advantage is not robust in the extended run.

Threats To Validity

The phase change is abrupt and scheduled. The keep-model agent does not receive the change signal, but the flush diagnostic does. This is useful for diagnosis but not temporally uniform as an algorithm.

The model is deterministic and tabular. This makes stale entries easy to identify because each model entry stores the phase in which it was learned. A stochastic environment would require uncertainty estimates, recency statistics, or prediction-error traces instead of a clean phase label.

The environment has one change. A stronger final planning proposal should add repeated changes, gradual drift, and stochastic transition changes. Those variants would test whether a freshness mechanism is merely tuned to one switch or can operate as an ongoing computation-allocation rule.

The current run uses a fixed learning rate and simple uniform model sampling for keep/flush conditions. The next study should compare search-control mechanisms directly rather than only changing the model contents.

Reviewer Critique And Response

Planning reviewer: a stationary Dyna result would be too familiar and would not answer an Alberta Plan continual-learning question. Response: the environment was changed into a continuing nonstationary gridworld with explicit post-change recovery windows and stale-backup diagnostics.

Continual-agent reviewer: flush-on-change is an oracle and violates the spirit of ordinary experience if treated as an algorithm. Response: the report now labels flush only as a diagnostic ceiling and avoids presenting it as a deployable solution.

Statistics reviewer: the first pilot had five seeds and a small grid, so the result could have been a noise artifact. Response: the current cited evidence is the 20-seed, 20000-step larger-grid extended run, and conclusions are limited to robust patterns: pre-change planning benefit and persistent stale-backup signal.

Sutton-style reviewer: the interesting question is not whether planning scores higher, but what knowledge is being backed up and whether it is still relevant. Response: the report foregrounds stale-backup rate and model freshness rather than only average reward.

Conclusion

Dyna Planning Budget is a useful independent precursor because it demonstrates a Core RL failure mode: planning budget and model freshness are distinct design variables. Planning helps when the model is fresh, but keeping a model in a changing stream can continue to consume computation on obsolete entries even when late reward recovers. The strongest next step is not another flush comparison; it is a realistic online freshness mechanism, which is developed in the integrated

Continual Dyna Model Aging proposal.

Reproduction

```
cd /mnt/shared-storage-user/yupeng/Core-RL

PYTHONNOUSERSITE=1 MPLCONFIGDIR=/mnt/shared-storage-user/yupeng/Core-RL/.mplconfig
/data/yupeng/conda_envs/core-rl/bin/python experiments/alberta_core_rl/scripts/run_experiment.py --config
experiments/alberta_core_rl/configs/dyna_planning_budget/config_extended.json

PYTHONNOUSERSITE=1 MPLCONFIGDIR=/mnt/shared-storage-user/yupeng/Core-RL/.mplconfig
/data/yupeng/conda_envs/core-rl/bin/python experiments/alberta_core_rl/scripts/plot_report_figures.py
--result-dir experiments/alberta_core_rl/results/dyna_planning_budget/20260709T024602Z_extended --kind
dyna-budget
```